GitHub, Operation And Demonstration Runbook

Repository: https://github.com/3351666087/desktop-tutorial

This document is designed for tutors or group members who want to verify that the demonstration is supported by a real codebase and repeatable operating procedure.

Repository Evidence

The repository contains:

Area	Path
Final Arduino UNO firmware	`UNO_Phase3_TelemetryPatch/`
Final ESP32 gateway firmware	`ESP32_3B_MQTT_Gateway/`
Web dashboard and local MQTT broker	`SmartClassroom_WebDashboard/`
Persistent AI backend	`smart_ai/ai_backend_service.py`
Runtime AI verification	`tools/check_ai_backend_runtime.py`
Delivery self-check	`tools/self_check_delivery.py`
Team documents and presentation	`docs/`
One-command launcher	`run_smartclassroom.py`
UNO flashing helper	`flash_uno.py`

One-Command Startup

```
python .\run_smartclassroom.py
```

This starts the PySide6 launcher, local web dashboard, MQTT broker and AI backend.

Dashboard URLs

Use	URL
Laptop browser	`http://localhost:3000`
Phone on same Wi-Fi	`http://<laptop-ip>:3000`
Phone microphone endpoint	`https://<laptop-ip>:3443`

The HTTPS page uses a local self-signed certificate for browser microphone permission during the lab demonstration.

Pre-Demo Verification

Run:

```
python .\tools\self_check_delivery.py --compile
python .\tools\check_ai_backend_runtime.py
```

Expected results:

```
Self-check: PASS 57 / WARN 0 / FAIL 0
AI runtime: ok true
whisperDevice: cuda
preferenceDevice: cuda
qwen model: qwen3.6-plus
```

Hardware Flashing

UNO

```
python .\flash_uno.py --sketch phase3
```

ESP32

Edit `ESP32_3B_MQTT_Gateway/credentials.h` for the current Wi-Fi and laptop MQTT broker IP. Then flash:

```
arduino-cli --config-file .\arduino-cli-esp32.yaml compile --fqbn esp32:esp32:esp32
.\ESP32_3B_MQTT_Gateway
arduino-cli --config-file .\arduino-cli-esp32.yaml upload -p COM6 --fqbn esp32:esp32:esp32
.\ESP32_3B_MQTT_Gateway
```

Demonstration Sequence

1. Open the dashboard and show live telemetry.
2. Trigger PIR/light behavior and show lamp state.
3. Show temperature cooling and fan PWM/RPM.
4. Trigger emergency button and show safety alarm priority.
5. Use manual mode to control fan/lamp/status lights.
6. Open the mobile HTTPS dashboard.
7. Speak a command such as "switch to manual mode and set the fan to maximum speed".
8. Show the fan changing physically and the MQTT log/ACK updating.
9. Show analytics and the model recommendation card.
10. Mention the GitHub repository for code and repeatability.

Typical Troubleshooting

Symptom	Likely cause	Fix
Dashboard shows waiting	ESP32 cannot connect to MQTT or no telemetry from UNO	Check laptop IP, `credentials.h`, Wi-Fi and UART wiring
ESP32 online but UNO offline	UART line or UNO firmware issue	Check UNO A5 to ESP32 GPIO16 divider and shared ground
Phone page opens but microphone fails	HTTP page or certificate permission	Use `https://<laptop-ip>:3443` and allow microphone
Qwen command fails	API key missing or network unavailable	Check ignored `SmartClassroom_WebDashboard/data/secrets.json`
Fan command does not change hardware	manual mode command not reaching UNO or relay/fan supply issue	Check MQTT ACK, ESP32 UART, UNO power and 12 V fan supply
Temperature looks wrong	LM35 supply/reference or ADC coupling	Power LM35 from ESP32 5 V and verify GPIO34 ADC route

Why GitHub Matters

The repository proves that the project is more than a one-time classroom setup. It provides source code, deployment scripts, wiring documents, test scripts and presentation materials. This supports maintainability, repeatability and technical credibility.